

基于 AMX 指令集的 Transformer 模型推理性能 优化

本教学案例旨在提供一个全面且深入的工程实践项目，专注于在中央处理器（CPU）环境下，如何基于高级矩阵扩展（Advanced Matrix Extensions, AMX）指令集优化深度学习模型的推理性能。案例以自然语言处理（NLP）领域广泛应用的 Transformer 模型为核心，详细阐述其架构及计算密集型特点。通过引导逐步实践，案例将展示如何配置 AMX 环境，并基于英特尔 PyTorch 扩展（IPEX）对 DistilBERT 模型（一个轻量化的 Transformer 模型）进行 BF16 和 INT8 量化优化。通过本案例将学习基准测试方法，分析不同优化策略对推理延迟、吞吐量及模型精度的影响，并探讨优化后的模型在文本分析、机器翻译、语音识别等不同应用场景中的实际效果和性能提升。本案例强调理论与实践相结合，旨在提升同学们的工程实践能力和解决复杂问题的能力。

第一部分、理论基础与核心概念

本部分旨在为后续的实践操作奠定坚实的理论基础。首先介绍在现代 CPU 上优化深度学习推理的背景和重要性，阐述为何需要专门的硬件加速技术。随后，深入剖析 AMX 指令集的架构、编程模型及其在 AI 计算中的优势。最后，回顾 Transformer 模型的关键组成部分，并重点分析其固有的计算瓶颈，为后续的 AMX 优化指明方向。

一、现代 CPU 上的深度学习推理优化

1.1 AI 高效推理的迫切需求

随着人工智能（AI）技术的飞速发展，深度学习模型在各个领域的应用日益

广泛，从图像识别、自然语言处理到推荐系统和自动驾驶，AI 正在深刻改变着我们的生产和生活方式。然而，这些先进模型的部署和高效运行，特别是推理（Inference）阶段，面临着严峻的挑战。推理是将训练好的模型应用于新数据以产生预测或输出的过程，与一次性的模型训练不同，推理是持续性的操作，其成本会随着服务请求量的增加而线性增长，因此推理效率成为衡量 AI 应用实用性的关键指标。

影响推理计算成本的关键因素包括模型大小（参数数量和激活值）、计算量（每次推理所需的浮点运算次数，FLOPs）以及内存访问效率。特别是对于像 Transformer 这样的大型模型，其庞大的参数数量和复杂的计算逻辑（如自注意力机制）对计算资源和内存带宽提出了极高的要求。

1.2 Transformer 等大型模型的挑战

Transformer 模型凭借其卓越的性能已成为自然语言处理（NLP）等领域的主流架构。然而，其强大的能力也伴随着巨大的计算开销。Transformer 模型通常包含数百万甚至数十亿的参数，导致巨大的内存占用和计算负载。其核心的自注意力机制（Self-Attention）虽然能够有效捕捉序列中的长距离依赖关系，但其计算复杂度随输入序列长度的平方增长，这使得处理长序列时计算成本急剧上升。这些特性共同导致了 Transformer 模型在推理时可能出现高延迟和高昂的部署成本，这对于需要在边缘设备上运行或对实时性有严格要求的应用场景构成了严峻挑战，同时也限制了在没有专用硬件加速器（如 GPU）环境下的广泛部署。

1.3 CPU 硬件加速简介：Advanced Matrix Extensions (AMX)

为了应对在通用计算平台上高效执行 AI 推理任务的需求，较新的至强（Xeon）处理器中推出了高级矩阵扩展（Advanced Matrix Extensions, AMX）技术。AMX 是一套针对 x86 指令集架构（ISA）的扩展，专为加速深度学习工作负载中核心的矩阵乘法运算而设计，这些运算直接在至强处理器上执行。AMX 的引入是提升 CPU 在 AI 推理领域竞争力的重要举措，旨在通过架构级加速显著提高吞吐量，从而在特定场景下减少对独立图形处理单元（GPU）等专用加速器的依赖。AMX 作为内置的 AI 加速器，已集成到第四代、第五代及第六代至强可扩展处理器中。

这种将 AI 加速能力直接构建到通用 CPU 中的策略，反映了硬件发展的一个重要趋势。对于许多 AI 推理任务，特别是那些对延迟敏感、负载具有间歇性，或者部署环境对成本、功耗及可用性有严格限制（例如，无法轻易获得或部署 GPU 的场景）的情况，基于 CPU 的 AMX 加速提供了一个极具吸引力的解决方案。AMX 的目标并非完全取代 GPU，而是在更广泛的 AI 工作负载范围内，特别是对于参数量在一定规模以下（例如，少于 200 亿参数的大语言模型）的模型，提供一种更经济、更易于获取的硬件加速选项，从而优化总体拥有成本（TCO）。理解这一战略背景，有助于认识到 AMX 不仅仅是一项技术特性，更是 AI 硬件普及化和 TCO 优化趋势的一部分，并学会在实际工程项目中评估 CPU+AMX 是否是比依赖 GPU 更合适的方案。

二、深入解析 Advanced Matrix Extensions (AMX)

2.1 AMX 架构：Tile 寄存器与 TMUL 单元

AMX 的核心架构由两个主要组件构成，它们协同工作以实现高效的矩阵运算：

Tile（瓦片）寄存器：AMX 引入了一组二维寄存器，称为“Tile”。这些 Tile 寄存器可以看作是 CPU 核心内部的高速缓存，用于存储从内存中加载的较大矩阵的子块（即“瓦片”）。例如，一些资料显示每个核心拥有 8 个 1KB 大小的 Tile 寄存器。通过将计算所需的数据块加载到 Tile 中，可以显著减少对主内存的频繁访问，从而缓解传统矩阵运算中常见的数据传输瓶颈。

TMUL (Tile Matrix Multiply, 瓦片矩阵乘法) 单元：TMUL 是一个专门的加速器引擎，或者说是一个计算单元，它附着在 Tile 寄存器上。TMUL 内部包含一个由多个融合乘加（Fused Multiply-Add, FMA）单元组成的阵列，这些 FMA 单元能够高效地并行处理 Tile 寄存器中存储的矩阵数据。

这种“Tile-TMUL”架构使得高吞吐量的矩阵乘法运算可以直接在 CPU 核心内部完成，极大地提升了计算效率。

2.2 AMX 编程模型

为了使开发者能够利用 AMX 的硬件能力，AMX 提供了一套相应的编程模型。

Tile 配置 (TILECFG): AMX 允许程序员通过 TILECFG 寄存器来配置 Tile 的具体参数，如每个 Tile 的行数 (rows) 和每行的字节数 (bytes_per_row)。这种灵活性使得开发者可以根据特定算法的需求调整 Tile 的“形状”，从而更自然地匹配算法的数据结构。

Tile 的配置信息通常通过一个特定的数据结构 (如 C 语言中的 `__tilecfg` 结构体) 在内存中定义，然后加载到 TILECFG 寄存器中。该结构体通常包含以下字段：

palette_id: 调色板 ID，用于指定 Tile 的配置方案。palette_id 为 1 是一个常见的配置，定义了 Tile 寄存器的布局 and 可支持的操作。

start_row: 起始行，通常为 0。

colsb: 一个数组，定义了最多 16 个 Tile 中每个 Tile 每行的字节数。

rows: 一个数组，定义了最多 16 个 Tile 中每个 Tile 的行数。

palette_id 的概念允许 AMX 支持多种不同的 Tile 配置和操作模式，为未来的扩展和特定优化提供了可能。

支持的数据类型: AMX 主要针对 BF16 (BFloat16) 和 INT8 (8 位整数) 这两种数据类型进行加速。

BF16 是一种 16 位浮点格式，它具有与 FP32 (32 位单精度浮点) 相同的指数位范围，但尾数位较少。这使得 BF16 在保持接近 FP32 的数值稳定性和表示范围的同时，能够将内存占用和计算量减半，非常适合深度学习中对精度要求不是极高但对性能敏感的场景。

INT8 是一种 8 位整型格式。通过将模型的权重和激活值量化到 INT8，可以进一步大幅降低模型大小、内存带宽需求，并利用专门的 INT8 计算指令 (如 AMX 中的 TMUL 对 INT8 的支持) 实现更快的推理速度。INT8 量化是提升模型推理性能的常用技术。

2.3 AMX 内置函数 (Intrinsics) 概览

为了让 C/C++ 等底层语言能够直接调用 AMX 指令，AMX 提供了一系列内置函数 (Intrinsics)。这些内置函数在编译时会被替换为相应的 AMX 硬件指令。主要的 AMX 内置函数可以分为以下几类：

Tile 配置类：

`_tile_loadconfig(cfg)`：从 `cfg` 指向的内存位置加载 Tile 配置信息到 TILECFG 寄存器。这是使用任何 Tile 操作之前必须执行的步骤。

`_tile_storeconfig(cfg)`：将当前的 Tile 配置信息存储到 `cfg` 指向的内存位置。

`_tile_release()`：释放当前活动的 Tile 配置。

Tile 数据加载/存储类：

`_tile_load(t, ld)`：从 `ld` 指向的内存位置加载数据到指定的 Tile 寄存器 `t`。加载的数据量和在 Tile 内的组织方式由当前 Tile 配置决定。

`_tile_store(st, t)`：将 Tile 寄存器 `t` 中的数据存储在 `st` 指向的内存位置。

`_tile_stream_load(t, ld)`：执行流式加载，暗示处理器加载的数据可能只使用一次，有助于减少缓存污染。

`_tile_zero()`：将所有 Tile 寄存器清零。

矩阵乘法类 (TMUL 操作)：

`_tile_dpbf16ps(c, a, b)`：执行 BF16 点积运算。它将 Tile `a` 和 Tile `b` (均为 BF16 数据) 进行矩阵乘法，并将结果累加到 Tile `c` (通常以单精度 FP32 格式累加，`ps` 代表 `packed single-precision`)。

`_tile_dpbf16sd(c, a, b)`：执行带符号 8 位整数 (`b`) 乘以带符号 8 位整数 (`b`) 的矩阵乘法，并将结果累加到带符号 32 位整数 (`sd`) 的目标 Tile `c`。LLVM IR 相关的讨论中也提到了类似的内部函数 `_tile_dpbf16sd_internal`。

其他整数类型变体，如 `_tile_dpbf16sud` (有符号 $\times$ 无符号 $\rightarrow$ 有符号累加), `_tile_dpbf16busd` (无符号 $\times$ 有符号 $\rightarrow$ 有符号累加), `_tile_dpbf16buud` (无符号 $\times$ 无符号 $\rightarrow$ 有符号累加)。

LLVM 编译器和相关的工具链也为 AMX 提供了支持，例如定义了用于表示 AMX Tile 数据的特定向量类型（如一个 1024 字节的 Tile 可能表示为 <256 x i32>）和 x86_amxtile 类型，以及与硬件指令一一对应的内部内置函数（如 _tile_load_internal）。

表 1: 关键 AMX 内置函数概览

内置函数名称 (Intrinsic Name)	简要描述	主要功能
_tile_loadconfig	加载 Tile 配置	初始化 AMX Tile 硬件参数
_tile_loadd	从内存加载数据到 Tile	将操作数移入 AMX Tile
_tile_dpbf16ps	BF16 点积累加到单精度	执行 BF16 矩阵乘法
_tile_dpbssd	带符号 INT8 点积累加到带符号 INT32	执行 INT8 矩阵乘法
_tile_stored	将 Tile 数据存储回内存	将计算结果写回内存

表 1 提供了一个关于 AMX 底层操作的快速参考，有助于理解在高级库（如 oneDNN 和 IPEX）背后，数据是如何被 AMX 硬件管理和处理的。

2.4 AMX 的硬件与软件生态

支持的 CPU：AMX 技术目前已在至强可扩展处理器的第四代（代号 Sapphire Rapids）、第五代（代号 Emerald Rapids）以及后续的第六代产品中得到支持。

操作系统要求：为了充分利用 AMX 功能，需要较新的 Linux 内核版本。通常要求 Linux 内核版本 5.16 或更高。主流的 Linux 发行版，如 CentOS Stream 8+、Ubuntu 22.04+ 等，在其较新的公共云镜像或本地安装中通常已包含对 AMX 的支持。

软件库：

oneAPI 深度神经网络库 (oneDNN)：oneDNN 是一个开源的高性能深度学习基础模块库，针对某些架构（包括 CPU 和 GPU）进行了深度优化，也包括对

AMX 的支持。主流的深度学习框架如 PyTorch 和 TensorFlow 在其 CPU 后端都利用 oneDNN 来实现加速。

英特尔 PyTorch 扩展 (IPEX): IPEX 是一个针对 PyTorch 框架的扩展库，它封装了针对某些硬件的特定优化，为 PyTorch 用户提供了更便捷的接口来利用 AMX 等硬件特性，尤其是在 BF16 和 INT8 推理方面。

其他工具: 如 llama.cpp（一个针对量化大语言模型的 C++ 推理框架）和 OpenVINO（推理优化工具套件）也提供了对 AMX 的支持或可以利用 AMX 的能力。

AMX 的利用呈现出一种分层的方式，这为开发者在性能和生产力之间取得了平衡。最底层是 AMX 硬件本身，其上是通过内置函数（Intrinsics）提供的低级编程接口，这给予了开发者极致的控制权，但直接使用这些接口开发大型应用会非常复杂。再往上，oneDNN 库将这些低级操作封装成更易于使用的深度学习基础模块（Primitives）。对于大多数应用开发者而言，通常会通过深度学习框架（如 PyTorch）间接使用 oneDNN，或者通过像 IPEX 这样的框架扩展库来更明确地调用和配置 AMX 优化。例如，PyTorch 默认使用 oneDNN 作为其 CPU 加速后端，因此在支持 AMX 的硬件上，许多操作可以自动获得 AMX 加速，无需用户手动干预。IPEX 则通过 `ipex.optimize()` 等函数进一步简化了在 PyTorch 中启用 AMX BF16 或 INT8 优化的过程。本教学案例将主要聚焦于 IPEX 层面，但了解其下的 oneDNN 和 AMX 内置函数对于深入理解和问题排查至关重要。

三、自然语言处理中的 Transformer 模型

3.1 核心架构组件

Transformer 模型自提出以来,已成为自然语言处理(NLP)领域的基石架构。其核心组件及其与矩阵运算的关联如下:

输入嵌入 (Input Embeddings) 与位置编码 (Positional Encoding): 输入文本首先被转换为词嵌入向量,然后加入位置编码向量以保留序列顺序信息。这些是模型的初始步骤,为后续的复杂计算提供输入。

自注意力机制 (Self-Attention Mechanism): 这是 Transformer 的核心。

查询 (Query, Q), 键 (Key, K), 值 (Value, V) 向量的生成: 每个输入词嵌入(或上一层的输出)会通过三个独立的线性变换(本质上是矩阵乘法)分别生成 Q, K, V 向量。这些线性变换的权重是模型学习得到的。

缩放点积注意力 (Scaled Dot-Product Attention): 注意力分数的计算涉及 Q 和 K 转置的点积 ($Q \cdot K^T$), 然后进行缩放(通常除以 d_k , d_k 是 K 向量的维度), 再通过 Softmax 函数归一化得到注意力权重。最后,将这些权重与 V 向量进行加权求和。这里的 $Q \cdot K^T$ 和后续与 V 的加权求和都是计算密集型的矩阵乘法操作。

多头注意力机制 (Multi-Head Attention): Transformer 并行地执行多个自注意力操作(称为“头”)。每个头使用独立的 Q, K, V 权重矩阵,学习输入序列的不同方面表示。各个头的输出随后被拼接(concatenate)起来,并通过另一次线性变换(矩阵乘法)得到最终的多头注意力输出。

前馈网络 (Feed-Forward Networks, FFN): 在每个编码器(Encoder)和解码器(Decoder)层中,多头注意力的输出会经过一个位置独立的前馈网络。FFN 通常由两个线性变换(矩阵乘法)和一个非线性激活函数(如 ReLU)组成。

编码器-解码器结构 (Encoder-Decoder Structure): 对于序列到序列任务(如机器翻译), Transformer 采用编码器-解码器架构。解码器除了包含自注意力和 FFN 外,还有一个编码器-解码器注意力机制,允许解码器在生成输出时关注输入序列的相关部分。

3.2 计算热点：识别矩阵乘法密集型操作

从上述架构分析可以看出，Transformer 模型中的计算瓶颈主要集中在大量的矩阵乘法运算上。具体而言：

Q, K, V 向量的生成（线性变换）。

注意力分数的计算中的 $Q \cdot K^T$ 。

注意力权重与 V 向量的加权求和。

多头注意力输出的最终线性变换。

前馈网络中的两个线性变换。

有研究明确指出，“自注意力算法，特别是其中的矩阵-矩阵乘法（MatMul）操作，需要大量的内存和计算复杂度”。MatMul 和 BatchMatMul 操作会消耗 BERT（一种 Transformer 编码器模型）大量的运行时间，因此优化 MatMul 和 BatchMatMul 是提升性能的关键。这些矩阵乘法密集型操作正是 AMX 指令集能够发挥其加速作用的主要目标。

3.3 关键 NLP 应用

Transformer 及其变体已成功应用于多种 NLP 任务：

文本分析 (如情感分类、文本分类): BERT、DistilBERT 等模型被广泛用于此类任务。本教学案例的实践部分将聚焦于此。

机器翻译 (NMT): Transformer 是神经机器翻译的奠基性架构。

语音识别 (ASR): Transformer 模型，如 Whisper，在自动语音识别系统中也展现出强大的性能。

Transformer 模型中矩阵乘法运算的普遍存在性，使其成为 AMX 技术的一个天然的、理想的优化对象。AMX 专为加速矩阵运算而设计，因此其应用于 Transformer 的各个计算核心模块是直接且显著有效的。通过明确理解这种联系：识别出 Transformer 中哪些部分是矩阵乘法密集型的，才能理解 AMX 为何能带来加速，以及在性能分析时应该关注哪些部分的提升。这充分证明了选择 Transformer 模型作为 AMX 优化案例的合理性。

第二部分、实践指南：基于 AMX 加速 Transformer 推理

本部分将理论知识付诸实践。首先将搭建一个支持 AMX 的实验环境，然后通过一系列引导性实验，使用英特尔 PyTorch 扩展（IPEX）对 DistilBERT 模型进行 BF16 和 INT8 精度的优化，并对每个阶段的性能进行基准测试和分析。

四、实验环境搭建

4.1 硬件要求与验证

CPU: 实验要求使用支持 AMX 指令集的至强处理器（第四代或更新版本）。

验证 AMX 支持: 在 Linux 系统中，可以通过以下命令检查 CPU 是否支持 AMX:

```
cat /proc/cpuinfo | grep amx
```

如果输出中包含 amx_bf16、amx_int8 等标志，则表明 CPU 支持 AMX。

4.2 软件栈安装与配置

一个正确配置的软件环境对于复现实验结果和发挥 AMX 性能至关重要。以下是推荐的软件栈及其安装步骤:

操作系统: 推荐使用较新版本的 Linux 发行版，内核版本需为 5.16+ (例如 Ubuntu 22.04 LTS)。

Python 环境: 建议使用 Conda 创建独立的 Python 环境，以避免包版本冲突。推荐 Python 3.8 或更高版本 (建议 Python 3.10)。

```
conda create -n amx_lab python=3.10 -y
conda activate amx_lab
```

PyTorch: 安装 CPU 版本的 PyTorch。需要注意 PyTorch 版本与 IPEX 版本的兼容性 (例如, PyTorch 2.0.x 通常对应 IPEX 2.0.x, PyTorch 2.1.x 对应 IPEX 2.1.x 等)。

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

英特尔 PyTorch 扩展 (IPEX): 安装 IPEX 库。

```
pip install intel-extension-for-pytorch
```

IPEX 扩展了 PyTorch，为至强硬件提供了优化，包括对 AMX 的支持。

oneAPI 深度神经网络库 (oneDNN): 通常作为 PyTorch (CPU 版本) 或 IPEX 的依赖自动安装。PyTorch 通过 oneDNN 调用优化的 CPU 计算核心。

HuggingFace 库:

transformers: 用于加载 Transformer 模型 (如 DistilBERT) 和分词器 (Tokenizer)。

datasets: 用于加载标准数据集 (如 GLUE SST-2)。

```
pip install transformers datasets
```

其他依赖: 如果使用 Jupyter Notebook 进行实验，还需要安装 notebook, ipykernel。绘图可能需要 matplotlib。

```
pip install notebook ipykernel matplotlib
```

表 2: 软件环境配置清单

组件 (Component)	推荐版本/细节	安装命令/备注
操作系统 (OS) / 内核 (Kernel)	Ubuntu 22.04 LTS 或类似发行版, Linux Kernel 5.16+ (建议 5.19+)	-
Python	3.10 (通过 Conda 管理)	conda create -n amx_lab python=3.10 -y
PyTorch	最新稳定 CPU 版本 (例如 2.1.x, 需与 IPEX 兼容)	pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
Intel Extension for PyTorch (IPEX)	最新稳定版本 (例如 2.1.x, 需与 PyTorch 兼容)	pip install intel-extension-for-pytorch

Intel oneAPI Deep Neural Network Library (oneDNN)	通常随 PyTorch/IPEX 自动安装	-
HuggingFace Transformers	最新稳定版本	pip install transformers
HuggingFace Datasets	最新稳定版本	pip install datasets
Jupyter (可选)	最新稳定版本	pip install notebook ipykernel matplotlib

4.3 使用 oneDNN Verbose 模式验证 AMX 功能

为了确认 AMX 是否在 PyTorch 运算中被实际使用，可以启用 oneDNN 的详细输出模式。oneDNN 是 PyTorch 在 CPU 上进行深度学习计算时依赖的底层优化库。

启用方法：

环境变量： 在运行 Python 脚本前设置环境变量 `ONEDNN_VERBOSE=1` (Linux/macOS) 或 `set ONEDNN_VERBOSE=1` (Windows CMD) 或 `$env:ONEDNN_VERBOSE = 1` (Windows PowerShell)。

Python 代码内： 使用 `torch.backends.mkldnn.verbose` 上下文管理：

```
import torch
#... 您的 PyTorch 代码...
with torch.backends.mkldnn.verbose(torch.backends.mkldnn.VERBOSE_ON):
    # 执行模型推理或相关操作
    model(inputs)
```

解读输出： 启用详细模式后，oneDNN 会在标准输出中打印其执行的每一个计算核心（primitive）的详细信息。需要关注以下关键信息来判断 AMX 是否被调用：

CPU ISA 信息： 输出的开头部分会包含 CPU 支持的指令集信息。查找类似 `cpu,isa:... amx_bf16... amx_int8...` 的行，确认 AMX BF16 和 INT8 支持已启用。

Primitive 执行信息： 每一行 `onednn_verbose,exec,...` 代表一个 oneDNN

primitive 的执行。关注 implementation 字段：

对于 BF16 操作，如果看到 `jit:avx512_core_amx_bf16` 或 `brg:avx512_core_amx_bf16` (通常用于 `matmul` 或 `convolution`)，则表明 AMX BF16 核心被使用。

对于 INT8 操作，如果看到 `jit:avx512_core_amx_int8` 或 `brg:avx512_core_amx_int8`，则表明 AMX INT8 核心被使用。

需要注意的是，某些情况下 IPEX 可能不通过 `oneDNN` 直接调用其自身优化的 BF16 融合核心，此时 `DNNL_MAX_CPU_ISA` 环境变量可能不影响这些特定路径，AMX 可能默认被 IPEX 使用。在本实验中，我们将主要依赖 `ONEDNN_VERBOSE` 来观察那些通过 `oneDNN` 执行的操作。

环境配置是高性能计算和人工智能实践中的第一个关键环节。特定版本的操作系统、内核、库（PyTorch、IPEX、Transformers）乃至编译器工具集的重要性。例如，Linux 内核版本要求 5.16+ 或 5.19+，IPEX 版本也与 PyTorch 版本紧密相关。要实现 AMX 等高级硬件特性的最佳性能甚至基本功能，高度依赖于正确配置的软件栈。不兼容性可能导致 AMX 未被使用，甚至出现运行时错误。因此，必须严谨细致地进行环境设置，通过 `ONEDNN_VERBOSE` 进行的验证步骤，对于确认设置符合 AMX 的需求至关重要。

五、分步实验指南：优化 DistilBERT 进行情感分析

本节将指导逐步完成对 DistilBERT 模型进行情感分析任务的优化实验。

模型选择：DistilBERT (distilbert-base-uncased-finetuned-sst-2-english)。选择该模型是因为它在性能和模型大小之间取得了较好的平衡，适合作为教学实验模型。它是一个常见的用于文本分类的 Transformer 模型。

任务：基于 SST-2 数据集进行情感分析（二分类：正面/负面）。

性能指标：

延迟 (Latency)：处理一个批次 (batch) 或单个样本所需的平均时间，单位为毫秒 (ms)。

吞吐量 (Throughput)：每秒处理的样本数量 (samples/sec)。

模型准确率 (Accuracy)：在 SST-2 验证集上评估模型的分类准确率。

基准测试方法：

使用 `time.perf_counter()` 精确计时代码块的执行时间。

在正式计时前，执行若干次“热身 (warm-up)”迭代，以消除首次运行时的 JIT 编译、缓存加载等开销对计时结果的影响。

对固定数量的迭代进行推理，并取平均性能指标，以获得更稳定的结果。

确保在不同优化阶段使用相同的批处理大小 (batch size) 以进行公平比较。

在推理过程中，使用 `with torch.no_grad()`：上下文管理器禁用梯度计算，以节省内存并加速推理。

5.1 基线性能测量 (FP32)

首先，我们需要测量未经任何 AMX 优化（即在 FP32 精度下）的原始 DistilBERT 模型的性能作为基准。

加载模型和分词器：

使用 HuggingFace transformers 库加载预训练的 distilbert-base-uncased-

finetuned-sst-2-english 模型及其对应的分词器。

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

model_name = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model_fp32 = AutoModelForSequenceClassification.from_pretrained(model_name)
model_fp32.eval() # 设置为评估模式
```

准备 SST-2 数据集：

使用 HuggingFace datasets 库加载 GLUE 基准测试中的 SST-2 数据集。

对句子进行分词处理，转换为模型可接受的输入格式。

创建一个 PyTorch DataLoader 用于批量加载验证集数据。

```
from datasets import load_dataset
from torch.utils.data import DataLoader

# 1. 加载 SST-2 数据集
sst2_dataset = load_dataset("glue", "sst2")

# 2. 定义分词函数
def tokenize_function(examples):
    return tokenizer(examples["sentence"], padding="max_length", truncation=True,
max_length=128)

# 3. 对数据集进行分词
tokenized_sst2 = sst2_dataset.map(tokenize_function, batched=True)
tokenized_sst2 = tokenized_sst2.remove_columns(["sentence", "idx"])
tokenized_sst2 = tokenized_sst2.rename_column("label", "labels")
tokenized_sst2.set_format("torch")

# 4. 创建 DataLoader (使用验证集进行基准测试)
batch_size = 32 # 可根据实际硬件调整
val_dataloader = DataLoader(tokenized_sst2["validation"], batch_size=batch_size)
```

基线推理循环与基准测试：

编写 Python 代码，遍历 val_dataloader，使用 FP32 模型进行推理，并测量延迟和吞吐量。同时，计算模型在 SST-2 验证集上的准确率。


```

import time
from torcheval.metrics import MulticlassAccuracy # 用于计算准确率

device = torch.device("cpu")
model_fp32.to(device)
accuracy_metric = MulticlassAccuracy(device=device)

total_time_fp32 = 0
total_samples_fp32 = 0
warmup_iterations = 5
actual_iterations = 20

# 热身
for i, batch in enumerate(val_dataloader):
    if i >= warmup_iterations:
        break
    input_ids = batch['input_ids'].to(device)
    attention_mask = batch['attention_mask'].to(device)
    labels = batch['labels'].to(device)
    with torch.no_grad():
        outputs = model_fp32(input_ids, attention_mask=attention_mask)

# 正式测试
for i, batch in enumerate(val_dataloader):
    if i >= actual_iterations:
        break
    input_ids = batch['input_ids'].to(device)
    attention_mask = batch['attention_mask'].to(device)
    labels = batch['labels'].to(device)

    start_time = time.perf_counter()
    with torch.no_grad():
        outputs = model_fp32(input_ids, attention_mask=attention_mask)
    end_time = time.perf_counter()

    total_time_fp32 += (end_time - start_time)
    total_samples_fp32 += input_ids.size(0)

    predictions = torch.argmax(outputs.logits, dim=-1)
    accuracy_metric.update(predictions, labels)

avg_latency_fp32 = (total_time_fp32 / actual_iterations) * 1000 # ms per batch
throughput_fp32 = total_samples_fp32 / total_time_fp32 # samples/sec
accuracy_fp32 = accuracy_metric.compute().item()

```

```
print(f"FP32 Baseline Performance:")
print(f"  Average Latency per batch: {avg_latency_fp32:.2f} ms")
print(f"  Throughput: {throughput_fp32:.2f} samples/sec")
print(f"  Accuracy: {accuracy_fp32:.4f}")
```

HuggingFace 上的 `distilbert-base-uncased-finetuned-sst-2-english` 模型报告的 dev set 准确率为 91.3% [1], Dataloop AI 上提到其 FP32 准确率为 0.9106 [2]。实验结果应接近此数值。

5.2 AMX 加速与 BFloat16 (BF16) 精度

接下来,我们将使用 IPEX 将模型转换为 BF16 精度,并利用 AMX 进行加速。

BF16 的原理: BF16 (BFloat16) 是一种 16 位浮点表示格式。与 FP16 (半精度浮点) 相比, BF16 保留了与 FP32 (单精度浮点) 相同的指数位数,这意味着它具有与 FP32 相似的动态范围,从而在转换时能更好地保持数值稳定性,减少溢出或下溢的风险。而其尾数位数较少,使得计算更快且内存占用更低。对于支持 BF16 的硬件 (如具备 AMX 指令集的 CPU), 使用 BF16 可以显著提升性能,同时通常只会带来微小的精度损失。

基于 IPEX 进行 BF16 优化:

导入 IPEX 库: `import intel_extension_for_pytorch as ipex`。

使用 `ipex.optimize()` 函数进行优化,并指定 `dtype=torch.bfloat16`。

在推理时,使用 `torch.cpu.amp.autocast()` 上下文管理器,它会自动将模型中的某些运算转换为 BF16 执行,以利用硬件加速。

```
import intel_extension_for_pytorch as ipex

# 重新加载原始 FP32 模型以进行 BF16 优化
model_to_bf16 = AutoModelForSequenceClassification.from_pretrained(model_name)
model_to_bf16.eval()
model_to_bf16.to(device)

# 使用 IPEX 进行 BF16 优化
```

```

optimized_model_bf16 = ipex.optimize(model_to_bf16, dtype=torch.bfloat16)

# 使用 torch.jit.trace 进一步优化 (可选但推荐)
# 需要一个示例输入来追踪模型
example_batch = next(iter(val_dataloader))
example_input_ids = example_batch['input_ids'].to(device)
example_attention_mask = example_batch['attention_mask'].to(device)

with torch.no_grad(), torch.cpu.amp.autocast():
    traced_model_bf16 = torch.jit.trace(optimized_model_bf16, (example_input_ids,
example_attention_mask), strict=False)
    traced_model_bf16 = torch.jit.freeze(traced_model_bf16)

# BF16 推理与基准测试
accuracy_metric_bf16 = MulticlassAccuracy(device=device)
total_time_bf16 = 0
total_samples_bf16 = 0

# 热身
for i, batch in enumerate(val_dataloader):
    if i >= warmup_iterations:
        break
    input_ids = batch['input_ids'].to(device)
    attention_mask = batch['attention_mask'].to(device)
    with torch.no_grad(), torch.cpu.amp.autocast(): # 确保 autocast 用于 BF16
        outputs = traced_model_bf16(input_ids, attention_mask=attention_mask)

# 正式测试
for i, batch in enumerate(val_dataloader):
    if i >= actual_iterations:
        break
    input_ids = batch['input_ids'].to(device)
    attention_mask = batch['attention_mask'].to(device)
    labels = batch['labels'].to(device)

    start_time = time.perf_counter()
    with torch.no_grad(), torch.cpu.amp.autocast(): # 确保 autocast 用于 BF16
        outputs = traced_model_bf16(input_ids, attention_mask=attention_mask)
    end_time = time.perf_counter()

    total_time_bf16 += (end_time - start_time)
    total_samples_bf16 += input_ids.size(0)

# 注意: traced_model_bf16 的输出可能直接是 logits, 也可能是一个元组

```

```

# 取决于原始模型的 forward 定义和 trace 的方式。
# 如果是 AutoModelForSequenceClassification, 输出是一个对象, logits 在 .logits 属性中
# 但 JIT trace 后, 输出结构可能改变。通常是 logits 张量。
# 此处假设 traced_model_bf16 直接返回 logits
if hasattr(outputs, 'logits'):
    logits = outputs.logits
else:
    logits = outputs # 假设直接是 logits

predictions = torch.argmax(logits, dim=-1)
accuracy_metric_bf16.update(predictions, labels)

avg_latency_bf16 = (total_time_bf16 / actual_iterations) * 1000 # ms per batch
throughput_bf16 = total_samples_bf16 / total_time_bf16 # samples/sec
accuracy_bf16 = accuracy_metric_bf16.compute().item()

print(f"\n\nIPEX BF16 Performance (with AMX if available):")
print(f"  Average Latency per batch: {avg_latency_bf16:.2f} ms")
print(f"  Throughput: {throughput_bf16:.2f} samples/sec")
print(f"  Accuracy: {accuracy_bf16:.4f}")

```

分析:

将 BF16 的性能指标（延迟、吞吐量、准确率）与 FP32 基线进行比较。预期在支持 AMX BF16 的硬件上会观察到显著的性能提升。

可设置 `ONEDNN_VERBOSE=1` 环境变量，并观察 `oneDNN` 的输出，查找是否有 `avx512_core_amx_bf16` 核心被调用的记录，以确认 AMX 硬件单元是否被用于 BF16 计算。

5.3 AMX 加速与 INT8 量化 (训练后静态量化 - PTSQ)

INT8 量化是另一种重要的模型优化技术，它可以进一步减小模型体积并加速推理。

INT8 量化原理： INT8 量化是指将模型的权重和/或激活值从浮点数（如 FP32 或 BF16）转换为 8 位整数。这极大地减少了模型的内存占用和数据传输带宽需求，并且在支持 INT8 计算的硬件（如 AMX INT8）上可以实现更快的运算速度。

静态量化与动态量化：

静态量化 (Static Quantization): 在模型推理前，通过一个“校准 (calibration)”过程来确定最佳的量化参数（如缩放因子 `scale` 和零点 `zero-point`）。校准过程需要小部分有代表性的数据集。静态量化通常能带来更好的性能，因为权重和激活值都预先量化好了。

动态量化 (Dynamic Quantization): 模型的权重是预先量化好的，但激活值是在推理过程中动态量化的。这种方法不需要校准数据集，但性能提升可能不如静态量化。本实验将采用静态量化，因为它通常能在 `AMX` 上获得更好的性能收益。

IPEX 静态量化工作流程: IPEX 提供了一套完整的 API 来支持 PyTorch 模型的静态量化。

定义量化配置 (QConfigMapping): 指定如何量化模型中的不同模块。IPEX 提供了默认配置。

准备模型 (ipex.quantization.prepare): 将原始的 `FP32` 模型和一个量化配置以及示例输入传递给 `prepare` 函数。该函数会在模型中插入“观察者 (observer)”模块，用于在校准阶段收集激活值的统计信息。

校准模型: 将有代表性的校准数据集（本例中使用 `SST-2` 验证集的一部分）输入到准备好的模型中。观察者模块会记录激活值的范围（如最小值、最大值）。

转换模型 (ipex.quantization.convert): 使用 `convert` 函数，根据校准阶段收集到的统计信息，将准备好的模型转换为真正的 `INT8` 量化模型。观察者模块会被替换为实际的量化/反量化操作以及量化后的层。

（可选）**JIT 追踪和冻结:** 为了部署，通常会将量化后的模型通过 `torch.jit.trace` 转换为 `TorchScript` 格式，并使用 `torch.jit.freeze` 进行优化。

动手实践：INT8 静态量化与重新基准测试：

```
from torch.ao.quantization import QConfigMapping
import intel_extension_for_pytorch.quantization as ipex_quant

# 重新加载原始 FP32 模型以进行 INT8 量化
```

```

model_to_int8 = AutoModelForSequenceClassification.from_pretrained(model_name)
model_to_int8.eval()
model_to_int8.to(device)

# 1. 定义量化配置
qconfig_mapping = ipex_quant.default_static_qconfig_mapping()

# 2. 准备模型进行校准
# 使用 val_dataloader 的一个批次作为 example_inputs
example_batch_for_prepare = next(iter(val_dataloader))
example_input_ids_for_prepare = example_batch_for_prepare['input_ids'].to(device)
example_attention_mask_for_prepare =
example_batch_for_prepare['attention_mask'].to(device)

# 注意：对于复杂的模型结构，example_inputs 可能需要更仔细地构造
# 对于 DistilBERT, 通常 (input_ids, attention_mask) 即可
prepared_model_int8 = ipex_quant.prepare(model_to_int8, qconfig_mapping,

example_inputs=(example_input_ids_for_prepare, example_attention_mask_for_prepare),
                inplace=False)

# 3. 校准模型
# 使用一部分验证集数据进行校准
num_calibration_batches = 10 # 可以根据数据集大小调整
print(f"\nStarting INT8 static quantization calibration using {num_calibration_batches} batches...")
with torch.no_grad():
    for i, batch in enumerate(val_dataloader):
        if i >= num_calibration_batches:
            break
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        prepared_model_int8(input_ids, attention_mask=attention_mask)
        if (i + 1) % 5 == 0:
            print(f"    Calibrated with batch {i+1}")
print("INT8 calibration finished.")

# 4. 转换模型为 INT8
with torch.no_grad(): # convert 也最好在 no_grad 下进行
    quantized_model_int8 = ipex_quant.convert(prepared_model_int8)

# 5. (可选但推荐) JIT 追踪和冻结
with torch.no_grad():
    traced_model_int8 = torch.jit.trace(quantized_model_int8,
(example_input_ids_for_prepare, example_attention_mask_for_prepare), strict=False)

```

```

    traced_model_int8 = torch.jit.freeze(traced_model_int8)
print("INT8 model traced and frozen.")

# INT8 推理与基准测试
accuracy_metric_int8 = MulticlassAccuracy(device=device)
total_time_int8 = 0
total_samples_int8 = 0

# 热身
for i, batch in enumerate(val_dataloader):
    if i >= warmup_iterations:
        break
    input_ids = batch['input_ids'].to(device)
    attention_mask = batch['attention_mask'].to(device)
    with torch.no_grad():
        outputs = traced_model_int8(input_ids, attention_mask=attention_mask)

# 正式测试
for i, batch in enumerate(val_dataloader):
    if i >= actual_iterations:
        break
    input_ids = batch['input_ids'].to(device)
    attention_mask = batch['attention_mask'].to(device)
    labels = batch['labels'].to(device)

    start_time = time.perf_counter()
    with torch.no_grad():
        outputs = traced_model_int8(input_ids, attention_mask=attention_mask)
    end_time = time.perf_counter()

    total_time_int8 += (end_time - start_time)
    total_samples_int8 += input_ids.size(0)

    if hasattr(outputs, 'logits'):
        logits = outputs.logits
    else:
        logits = outputs

    predictions = torch.argmax(logits, dim=-1)
    accuracy_metric_int8.update(predictions, labels)

avg_latency_int8 = (total_time_int8 / actual_iterations) * 1000 # ms per batch
throughput_int8 = total_samples_int8 / total_time_int8 # samples/sec
accuracy_int8 = accuracy_metric_int8.compute().item()

```

```
# 获取模型大小 (以 MB 为单位)
torch.jit.save(traced_model_int8, "temp_quantized_model.pt")
import os
model_size_int8_mb = os.path.getsize("temp_quantized_model.pt") / (1024 * 1024)
os.remove("temp_quantized_model.pt")

print(f"\nIPEX INT8 Static Quantization Performance (with AMX if available):")
print(f"  Average Latency per batch: {avg_latency_int8:.2f} ms")
print(f"  Throughput: {throughput_int8:.2f} samples/sec")
print(f"  Accuracy: {accuracy_int8:.4f}")
print(f"  Model Size: {model_size_int8_mb:.2f} MB")
```

性能分析：

将 INT8 模型的性能指标（延迟、吞吐量、准确率、模型大小）与 FP32 基线 和 BF16 结果进行比较。预期会观察到进一步的性能提升和显著的模型体积减小。

重点评估准确率的变化。讨论在当前应用场景下，这种准确率的损失是否可以接受，以换取性能的提升。Dataloop AI 的数据显示，对于 DistilBERT SST-2 模型，PyTorch INT8 量化后的准确率为 0.9037，而 FP32 为 0.9106，存在一定的精度下降 [2]。这也符合相关的“盲目应用量化可能导致显著的准确率下降”研究结论。

可再次使用 `ONEDNN_VERBOSE=1` 观察输出，查找是否有 `avx512_core_amx_int8` 核心被调用的记录。

在优化过程中，可体会到“天下没有免费的午餐”这一原则。虽然 BF16 优化在 AMX BF16 硬件支持和 IPEX 的良好处理下，通常能在保持较高精度的同时带来速度提升，但 INT8 量化则引入了更显著的数据表示变化。如前所述，DistilBERT SST-2 的 INT8 量化确实可能导致准确率下降。这清晰地展示了性能提升与潜在精度损失之间的权衡。必须认识到，优化不仅仅是让程序运行得更快，更是在特定应用需求下，对性能、准确率、模型大小、开发复杂度等多个因素进行综合考量和权衡。实验应鼓励批判性地分析每一步优化所带来的结果，而不仅仅是观察速度的提升。

六、结果分析与讨论

6.1 性能指标对比分析

完成上述所有实验步骤后，应将收集到的数据整理成如下表格，以便进行清晰的对比分析。

表 3：DistilBERT-SST2 推理性能对比 (示例)

优化类型	平均延迟 (ms/batch)	吞吐量 (samples/sec)	模型大小 (MB)	SST-2 验证集准确率 (%)	相对 FP32 加速比
基线 (FP32)	实验数据	实验数据	原始大小	实验数据 (应接近 0.91)	1.0x
IPEX BF16 (AMX)	实验数据	实验数据	略小于 FP32	实验数据	实验数据
IPEX INT8 PTSQ (AMX)	实验数据	实验数据	显著减小	实验数据 (可能略降)	实验数据

此表格是实验结果的核心，它量化了不同 AMX 优化技术对具体 NLP 任务的影响，能够直观地比较各种优化策略的优劣和权衡。同时，建议使用条形图等可视化方式展示延迟、吞吐量和准确率的对比结果，使分析更加直观。

6.2 AMX 如何加速特定 Transformer 层

观察到的性能提升与 Transformer 模型的架构密切相关（参见第 3.2 节）。AMX 主要通过加速矩阵乘法运算来提升 Transformer 的推理速度。在 Transformer 模型中，以下部分的计算密集型矩阵乘法是 AMX 加速的主要受益者：

自注意力机制 (Self-Attention):

Q, K, V 投影的线性变换。

注意力得分计算中的 $Q \cdot K^T$ 。

V 值基于注意力权重的加权求和。

多头注意力 (Multi-Head Attention):

每个注意力头的独立计算（同上）。

多个头输出拼接后的最终线性变换。

前馈网络 (Feed-Forward Network, FFN):

FFN 中的两个大规模线性变换。

AMX 的 `_tile_dpbf16ps`（用于 BF16）和 `_tile_dpbssd` 或类似指令（用于 INT8）能够高效执行这些矩阵运算。BERT 模型优化的案例表明，AMX 使得更深层次的算子融合成为可能，例如将 QKV 矩阵乘法的权重合并，从而减少内存访问并提升计算效率 [3]。这种融合优化进一步发挥了 AMX Tile 架构的优势，即将多个操作合并，使得中间数据可以保留在 Tile 寄存器或 CPU 缓存中，供后续操作使用，减少了与主存的交互。

6.3 INT8 量化的准确率-性能权衡评估

INT8 量化通常能带来最显著的性能提升和模型压缩效果，但也最有可能影响模型准确率。

准确率变化：应仔细分析 INT8 量化后模型的准确率。与 FP32 基线相比，准确率是否有所下降？如果下降，幅度有多大？例如，对于 DistilBERT SST-2 模型，有报告称 INT8 量化（PyTorch）后的准确率为 0.9037，而 FP32 为 0.9106 [2]。这种幅度的下降在某些应用中可能是可以接受的，但在其他对精度要求极高的场景则可能不行。

校准数据集的重要性：再次强调，静态量化的效果高度依赖于校准数据集的质量和代表性。如果校准数据与实际推理数据的分布差异较大，可能会导致量化参数选择不佳，从而影响准确率。

其他量化策略：虽然本实验仅涵盖了训练后静态量化 (PTSQ)，但应了解还存在其他量化技术，如训练后动态量化 (PTDQ) 和量化感知训练 (Quantization-Aware Training, QAT)。QAT 在训练过程中就引入了模拟量化的操作，通常能够获

得比 PTSQ 更好的准确率，但需要重新训练模型，成本较高。

硬件的潜力需要通过软件来释放，但硬件本身也设定了性能的上限。整个实践环节都依赖于 IPEX 和底层的 oneDNN 将 PyTorch 的操作转换为 AMX 可执行的代码。如果没有这些软件层，直接通过 AMX 内置函数编程将极为复杂。众多资料都强调了 oneDNN 和框架集成（如 PyTorch、TensorFlow）在利用 AMX 方面所扮演的关键角色。然而，最终的性能增益仍然受到 AMX 硬件自身能力的限制（例如 TMUL 单元的数量、Tile 大小、时钟频率等）。应理解高性能计算中硬件和软件之间的这种共生关系。像 AMX 这样的硬件进步需要相应的软件创新才能变得易于访问和有效。同时，也应认识到，软件优化可以帮助接近硬件的峰值性能，但硬件本身定义了这个峰值。

第三部分、总结与展望

本部分将实验所学扩展到更广泛的背景，讨论 AMX 优化的 Transformer 模型在其他 NLP 任务中的应用，并展望 CPU AI 加速的未来。

七、总结与展望

7.1 关键学习要点回顾

本教学案例包含了以下关键内容：

理解了 AMX 的核心架构（Tile 和 TMUL）及其在加速矩阵运算中的作用。

识别了 Transformer 模型中的主要计算热点，特别是自注意力和前馈网络中的矩阵乘法。

通过实践掌握了使用 Intel Extension for PyTorch (IPEX) 对 Transformer 模型（DistilBERT）进行 BF16 和 INT8（静态量化）优化的具体步骤。

学会了如何进行性能基准测试，并分析不同优化策略在延迟、吞吐量、模型大小和准确率之间的权衡。

7.2 在其他潜在 NLP 场景中的应用

本案例基于 AMX 和 IPEX 对 DistilBERT 进行 BF16 和 INT8 优化，其原理和方法具有广泛的适用性，可以推广到其他基于 Transformer 的模型和更多样的 NLP 任务中。Transformer 架构的核心计算模式（自注意力中的矩阵乘法、前馈网络中的矩阵乘法）在各种衍生模型中是共通的。由于 AMX 专为加速这些基础的矩阵运算而设计，因此其优化效果可以惠及众多 Transformer 应用。

文本分析 (从情感分析到更广泛的分类任务):

本实验中对 DistilBERT 进行情感分析的优化方法，可以直接应用于其他文本分类任务，如主题分类、垃圾邮件检测、意图识别等，只要它们采用的是 BERT 类或类似的 Transformer 编码器模型。

BERT 推理优化的案例 [3]以及相关的 BERT 调优指南 [4]提供了真实世界的例证，并展示了更深层次的优化策略，如算子融合，这些策略与 AMX 的能力

相辅相成。

机器翻译 (NMT):

神经机器翻译模型（如经典的 Transformer "Attention Is All You Need" 模型及其后续变种）同样以 Transformer 的编码器-解码器结构为核心，其中包含大量的矩阵乘法运算。

AMX 可以有效加速这些计算瓶颈，从而提升翻译模型的推理速度，降低服务延迟。Numenta 的案例研究表明，在第四代至强处理器上使用 AMX 对 BERT-Large 进行优化，获得了显著的吞吐量提升，这对于 NMT 模型的编码器部分同样具有参考意义 [5]。

语音识别 (ASR):

现代自动语音识别系统，特别是端到端模型，越来越多地采用 Transformer 架构（例如 OpenAI 的 Whisper 模型就是一个 Transformer 编码器-解码器模型）。

在采用 AMX BF16 的第五代至强处理器上，实时语音识别推理性能（以 RNN-T 模型为例，该模型也可能包含 Transformer 组件或类似的稠密矩阵运算）相比于第三代至强处理器的 FP32 性能提升高达 10.7 倍 [6]。这表明 AMX 对于提升 ASR 模型的推理效率具有巨大潜力。

可取得的预期性能如下：

Llama2 7B (大语言模型): 在第四代至强处理器上启用 AMX 后，第二词元平均延迟提升高达 40%；结合 INT8 量化可以进一步提升吞吐量 [7]。

GPT-J (大语言模型): 在第四代至强处理器上基于 AMX 进行推理，在 MLPerf Inference 基准测试中表现良好 [8]。

通用 AMX 性能: 对于 Llama 3.2B 模型（Q4_K_M 或 Q8_0 量化），启用 AMX 的 CPU 推理吞吐量可达 57 tokens/sec，而未启用 AMX 时为 28 tokens/sec (提升约 2 倍)；若采用更低比特的 4-bit 量化，则可达到 80 tokens/sec [9]。

AMX vs. AVX512: 在 oneDNN 层面，AMX 的某些操作性能比仅使用 AVX512

快 3.5 至 5 倍，并且功耗更低 [10]。

这些数据和应用案例共同说明，本教学案例中的 DistilBERT 优化技能是具有高度可迁移性的。通过识别模型中的矩阵乘法热点并应用 AMX 赋能的优化技术（如 BF16、INT8 量化），可以广泛适用于各种 Transformer 模型和 NLP 任务，从而使 AMX 技术更具通用性和实用价值。

7.3 进一步探索方向

为了巩固所学知识并进行更深入的探索，建议考虑以下项目或研究方向：

模型扩展：将本案例中的优化方法应用于其他 Transformer 模型，如 BERT-base、RoBERTa 或其他针对特定 NLP 任务（如问答、摘要）的模型。

动态量化探索：尝试使用 IPEX 实现并评估动态量化（PTDQ）在 DistilBERT 或其他模型上的性能和精度表现，并与静态量化进行对比。

校准策略研究：深入研究不同校准数据集大小、数据选取策略对静态量化后模型准确率和性能的影响。

底层探索 (高级)：尝试使用 AMX 内置函数（Intrinsics）为一个极小规模矩阵运算（例如单个小型矩阵乘法）编写 C++ 或汇编代码，以更直观地理解 AMX 的工作方式。

应用级性能分析：选择一个简单的、完整的 NLP 应用流程（例如，一个基础的机器翻译流水线或一个简单的语音命令识别系统），集成 AMX 优化后的 Transformer 模型，并对整个应用的端到端性能进行分析和瓶颈定位。

探索其他 AMX 应用场景：研究 AMX 在计算机视觉（如 ViT 模型中的矩阵运算）、推荐系统等其他 AI 领域的应用潜力和优化方法。

高性能计算和人工智能领域的技术发展日新月异。硬件的快速迭代和软件的持续更新（如 IPEX 的版本演进，oneDNN 的优化）意味着优化技术和最佳实践并非一成不变。诸如软件成熟度和实现最优性能的复杂性等挑战也表明，该领域的工程师必须不断更新其知识和技能。本教学案例提供的是当前技术的一个快照，是相关技术的基础。本节的展望部分旨在引导同学关注相关资源和研究领域，以便在快速发展的 AI 加速技术浪潮中保持前沿。

7.4 总结

本工程教学案例详细阐述了在 CPU 环境下基于 AMX 指令集优化 Transformer 模型推理过程的核心理论、关键技术和实践方法。通过对 AMX 架构、Transformer 模型计算特性以及英特尔 PyTorch 扩展 (IPEX) 优化工具的深入学习, 结合针对 DistilBERT 模型在 SST-2 情感分析任务上的 BF16 和 INT8 量化优化实验, 能够:

深刻理解 AMX 的加速原理: 掌握 AMX Tile 架构和 TMUL 单元如何通过加速核心的矩阵乘法运算来提升深度学习性能, 特别是在 BF16 和 INT8 数据类型上的优势。

熟练应用 IPEX 进行模型优化: 掌握使用 IPEX 对 PyTorch 模型进行 `ipex.optimize()` (针对 BF16) 以及完整的静态量化流程 (`prepare`, 校准, `convert` 针对 INT8) 的实际操作技能。

掌握性能评估与分析方法: 学会使用标准工具和方法对优化前后的模型进行延迟、吞吐量、准确率和模型大小等关键指标的基准测试, 并能对结果进行量化分析和权衡。

认识到软硬件协同的重要性: 理解先进硬件特性 (如 AMX) 的潜力需要通过相应的软件栈 (如 oneDNN, IPEX, 编译器) 来释放, 同时也认识到硬件是性能的天花板。

具备将优化技术推广应用的能力: 理解 AMX 对 Transformer 模型核心计算模块的普适性加速效果, 从而能将所学知识和技能迁移到更广泛的 NLP 任务 (如机器翻译、语音识别) 和其他 Transformer 架构中。

本案例不仅旨在传授具体的技术细节, 更重要的是培养在面对复杂工程问题时的分析能力、实践能力和创新思维。通过亲自动手完成从环境搭建、模型优化、性能测试到结果分析的全流程, 真正做到“学以致用”, 为未来从事 AI 系统优化、高性能计算等相关领域的研究和工作打下坚实的基础。随着 CPU 内置 AI 加速技术的不断发展, 掌握如 AMX 这样的核心技术将使未来的职业生涯中更具竞争力。

5. 案例教学建议

(1) 案例引入: 可以从当前 AI 应用对推理性能的迫切需求以及 Transformer 等大模型带来的挑战入手, 引出 CPU 硬件加速技术 AMX 的重要性。

(2) 课时分配 (建议):

- 理论基础与核心概念讲解: 2-3 学时。
- 实验环境搭建与验证: 1 学时 (可课前准备)。
- 分步实验 (FP32, BF16, INT8): 3-4 学时 (鼓励学生动手实践)。
- 结果分析与讨论: 1-2 学时。

(3) 教学讲授方式:

- 理论讲授与实践操作相结合。
- 引导学生按照案例中的步骤进行实验, 观察和记录实验数据。
- 鼓励小组讨论, 分析不同优化策略的效果和原因。

(4) 课堂讨论总结:

- 对比分析 FP32、BF16、INT8 三种情况下的性能指标和准确率, 讨论性能提升与精度损失之间的权衡。
- 讨论 AMX 技术在不同 NLP 任务和 Transformer 变体中的适用性。
- 启发学生思考进一步的优化方向和开放性问题。

6. 开放性问题

本案例的“进一步探索方向”部分提供了一些开放性问题和研究方向, 可供学生深入思考和实践:

1. 将本案例中的优化方法应用于其他 Transformer 模型 (如 BERT-base, RoBERTa) 或其他 NLP 任务 (如问答、摘要)。
2. 尝试使用 IPEX 实现并评估动态量化 (PTDQ) 在 DistilBERT 或其他模型上的性能和精度表现, 并与静态量化进行对比。
3. 深入研究不同校准数据集大小、数据选取策略对静态量化后模型准确率和性能的影响。
4. (高级) 尝试使用 AMX 内置函数为一个极小规模矩阵运算编写 C++ 或汇编代码, 以更直观地理解 AMX 的工作方式。

5. 选择一个简单的、完整的 NLP 应用流程，集成 AMX 优化后的 Transformer 模型，并对整个应用的端到端性能进行分析和瓶颈定位。
6. 研究 AMX 在计算机视觉（如 ViT 模型）、推荐系统等其他 AI 领域的应用潜力和优化方法。

7. 附件材料

案例中包含了详细的实验步骤和方案，另有案例配套的讲授 PPT、讲授视频和程序代码可供参考。

参考资料列表：

- [1] DistilBERT base uncased finetuned SST-2, <https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english>
- [2] Distilbert Base Uncased Finetuned Sst 2 English Int8 Static Inc, https://dataloop.ai/library/model/intel_distilbert-base-uncased-finetuned-sst-2-english-int8-static-inc/
- [3] Optimizing Machine Learning (ML) Models with Intel Advanced Matrix Extensions (Intel AMX), <https://www.intel.com/content/dam/www/central-libraries/us/en/documents/2022-12/optimizing-ml-models-with-amx-brief.pdf>
- [4] Tuning Guide for BERT-based AI inference with Intel Advanced Matrix Extensions, <https://www.intel.com/content/www/us/en/developer/articles/guide/bert-ai-inference-amx-4th-gen-xeon-scalable.html>
- [5] Numenta and Intel Deliver Cost-Effective, Powerful Inference Performance, https://cdrdv2-public.intel.com/788115/Numenta%20Case%20Study_final%20updated%20Nov%202023.pdf
- [6] What Is Intel Advanced Matrix Extensions (Intel AMX)?, <https://www.intel.com/content/www/us/en/products/docs/accelerator-engines/what-is-intel-amx.html>
- [7] Deploy and Scale Generative AI in Enterprises with Intel AMX and Lenovo, <https://lenovopress.lenovo.com/lp2190.pdf>
- [8] MLPerf Inference 3.1 on Dell PowerEdge Server with Intel 4th Generation Xeon CPU, <https://infohub.delltechnologies.com/p/mlperf-tm-inference-3-1-on-dell-powerededge-server-with-intel-r-4th-generation-xeon-r-cpu/>
- [9] Intel AMX Enables High-Efficiency CPU Inference for AI Workloads, <https://openmetal.io/resources/blog/intel-amx-ai-inference-performance/>
- [10] Intel Advanced Matrix Extensions [AMX] Performance With Xeon Scalable Sapphire Rapids, <https://www.phoronix.com/review/intel-xeon-amx/3>